Secure Embedded Water Tank Controller

Finley Burns
CYCS 3240
Final Report
Prepared November 10, 2026

***Abstract*—I built a C17 controller for one simulated water tank, level sensor, and inlet valve. The controller uses two thresholds, closes the inlet on invalid or stale readings, and accepts authenticated serial commands to change its settings. I tested the shared code natively on Windows, macOS, and Linux, using both x86-64 and ARM64 hosts. I also ran the actual Cortex-M4 firmware in Renode. Eighteen native tank runs passed their sample-by-sample checks. A separate 600-second ARM feedback run stayed between 29.9% and 70.1%. During three serial-load tests, the firmware kept its 100 ms schedule and closed the valve on the injected sensor fault. The results establish the behavior of this simulation. They do not establish physical-device timing, protected key storage, or replay prevention.**

***Index Terms*—C17, embedded systems, input validation, HMAC, simulation**

I. Project Description

I chose a water tank because the controller has a clear job: admit water when the level is low and stop admitting it when the level is high. Operator commands should change the operating thresholds without bypassing the input checks or stopping normal control.

The project began with Rust. I switched to C17 because the course requires C or C++, while keeping Renode and the STM32F4 Cortex-M4 target. The scope is one tank, one sensor, one inlet valve, and a serial interface. I used files for the test inputs and kept the shared decisions separate from the board code.

My goals were repeatable control, rejection of malformed or unauthorized settings, continued control under serial load, and a reproducible evaluation. The implementation and its tests are in the accompanying files and GitHub repository [1]. The November documents are earlier design snapshots; this report describes the later implementation.

II. Design and Implementation

A. Valve Decisions

Levels use basis points: 0 is empty and 10000 is full. The default low and high thresholds are 3000 and 7000. A valid configuration satisfies $0 \leq low < high \leq 10000$. The inlet opens strictly below the low threshold and closes strictly above the high threshold. Between the thresholds and at either endpoint, it keeps its previous state. This hysteresis avoids switching around a single target.

The controller starts closed with an invalid initial sample. An invalid flag, out-of-range level, or sample age greater than 300 ms closes the inlet and sets a fault. Age 300 is accepted; age 301 is rejected. Unsigned time subtraction handles the 32-bit timer wrap for the bounded intervals used here. A future timestamp is rejected. A valid reading clears the fault and resumes the same threshold rule.

The firmware uses a 1 ms SysTick counter and runs control every 100 ms. If the elapsed interval reaches two complete periods, it closes the valve and records a missed deadline. It does not issue a burst of catch-up updates. Host tests also exercise 50 ms and 500 ms control periods.

B. Firmware and Sensor Interface

The board code uses USART2 and PD12, following the STM32F4 register layout [2]. PD12 represents the commanded valve state. Startup assembly installs the reset, SysTick, and USART2 interrupt handlers. The linker reserves the first 32 bytes of RAM for the sensor record; normal program data begins at address 0x20000020.

The adapter publishes a sequence number, a 32-bit level, a validity flag, and a timestamp. It writes an odd sequence while updating the fields and an even sequence when finished. Firmware accepts the record only when both sequence reads agree and are even. It checks the full-width level before narrowing it. This prevents a value such as 68036 from becoming an apparently valid 2500 after conversion.

I used the planned RAM-record route instead of an ADC. The firmware executes on Renode's Cortex-M4 model, and the adapter reads the actual emulated GPIO output [3]. The platform description sets SysTick to 16 MHz to match the firmware's clock setup. A Python console adapter controls Renode directly; this implementation does not require Robot Framework, Cygwin, or WSL.

C. Serial Input and Authentication

A command has the form `SET,low,high,tag` followed by LF. Numbers are canonical decimal without signs, spaces, or leading zeroes. The tag has exactly 64 lowercase hexadecimal digits. HMAC-SHA256 covers the bytes `SET,low,high`, excluding the comma before the tag and the newline.

I used Mbed TLS 3.6.7 for HMAC and its constant-time comparison function [4]. A 4096-byte static allocator supplies the library's working memory. The release archive hash and upstream license are included. The public demonstration key is the byte sequence 00 through 1f. It is test data, not protected deployment credentials.

The receive queue holds 256 bytes, and the line buffer allows 128 bytes plus its terminator. The main loop consumes at most 32 bytes and one completed line per tick. It checks format and conversion bounds, verifies the tag, validates both thresholds, and then installs both settings together. Rejected commands leave the settings unchanged. A newly accepted configuration affects the next control tick.

Malformed or oversized lines are discarded through LF. Queue overflow clears queued and partial input and requires another LF before parsing resumes. A partial line expires when its age exceeds 500 ms. That age begins when the main loop processes the first byte, rather than at physical UART arrival. The 500 ms sampling sweep tests control and the model; it does not claim usable command reception with this unchanged timeout.

The firmware masks interrupts during its bounded control/serial service to protect the shared queue on the single-core target. The control decision is calculated before serial work, and GPIO is written when that bounded service returns. This is a small ordering difference from the original pseudocode, which placed the GPIO write immediately before serial parsing. The emulated load tests check the resulting behavior; they are not a hardware worst-case execution-time proof.

D. Secure Programming Choices

I checked integers before multiplication or narrowing, checked file and process failures, bounded buffers, and preserved exact input bytes. File readers reject embedded NUL and oversized records. Numeric parsing checks range, conversion errors, and trailing fields. The build treats warnings in project C code as errors. These choices follow the concerns covered by CERT C [5].

HMAC provides no encryption or replay prevention in this project. A correctly signed earlier command can restore old settings. The host and emulator are trusted. A stuck-open valve is modeled for testing, but firmware has no independent flow sensor or actuator-feedback alarm.

E. Tank Model and Interfaces

The separate C model uses signed 64-bit microliters and retains fractional flow remainders. It records the raw next volume and any shortage or overflow before clamping the volume to the tank's physical range. Its update is

$$V_{n+1} = V_n + (u_n Q_{in} - Q_{out})\Delta t, \tag{1}$$

where u_n is the actual modeled valve state. This can differ from the GPIO command when a test forces the valve open.

The default capacity is 100 L, the initial volume is 50 L, the inlet supplies 2 L/s, and normal demand is 1 L/s. Python reads the scenario files, applies events, sends samples to the controller, collects the valve output, and advances the C model. An independent Python reference uses exact rational arithmetic and the threshold specification. Every sample is compared, so an incorrect intermediate decision cannot hide behind a correct final volume.

III. Testing

A. Host Cases

The original suite passed 69 independent control cases, an 18-step stateful sequence, 23 parser files, and three invocation-error checks. One parser file deliberately contains an incorrect expected result; the runner must return 1 for that mismatch. Malformed input returns 2 and successful input returns 0.

The later suite adds 36 runtime scenarios. These include sensor age and recovery, timer wrap, missed deadlines, next-tick configuration use, queue capacity and overflow recovery, partial-line expiry, invalid commands, line lengths of 127, 128, 129, and 1024 bytes, replay, and 1000 consecutive authenticated updates. The valid command and its modified or wrongly signed counterparts are separate file inputs.

Five HMAC checks passed. Two use the published RFC 4231 values [6]; three use Python's separate HMAC implementation, including an empty key/message and a key longer than the SHA-256 block size. Ten additional malformed-file checks cover the new host readers. Model unit checks cover signed underflow, overflow, ordinary flow, and accumulated fractional microliters.

B. Native Tank Runs

I ran the normal 600-second case three times. The demand case changes demand to 0.5 L/s at 200 seconds and 1.5 L/s at 400 seconds, also repeated three times. Separate cases apply 3 L/s demand, force the valve open at 10 seconds, and run for 3600 seconds. The 50, 100, and 500 ms periods each have three 600-second repetitions. This gives 18 runs. Each repeated scenario produced identical CSV hashes.

C. ARM Execution and Load

The actual Cortex-M4 ELF passed 15 initial checks for threshold behavior, stale and invalid samples, torn records, wide values, valid commands, and invalid tags. I then ran three 60-second load modes: no commands, one valid command per second, and a malformed flood from 20 to 30 seconds. The adapter injected 100-byte bursts each 100 ms, averaging 1000 bytes/s. Each mode injects an invalid sensor at its 25-second tick. These tests add 1800 observed control ticks.

The GPIO closed on the fault tick in every mode, and the deadline counter stayed at zero. The full-queue counter recorded 1200 rejected incoming bytes. Recovery also discarded queued data. The arrival rate exceeded the 320-byte/s service budget. After a newline resynchronized the parser, a valid command was accepted. The final accepted-command count was 62 across the combined test sequence.

A separate 600-second closed-loop test connected the C model to the real emulated GPIO. It checked 6001 samples after the initial firmware startup interval. The model advances for exactly 600 seconds; firmware timestamps include the first 100 ms startup tick. I ran the longer, abnormal-flow, and sampling-sweep scenarios natively, not through Renode.

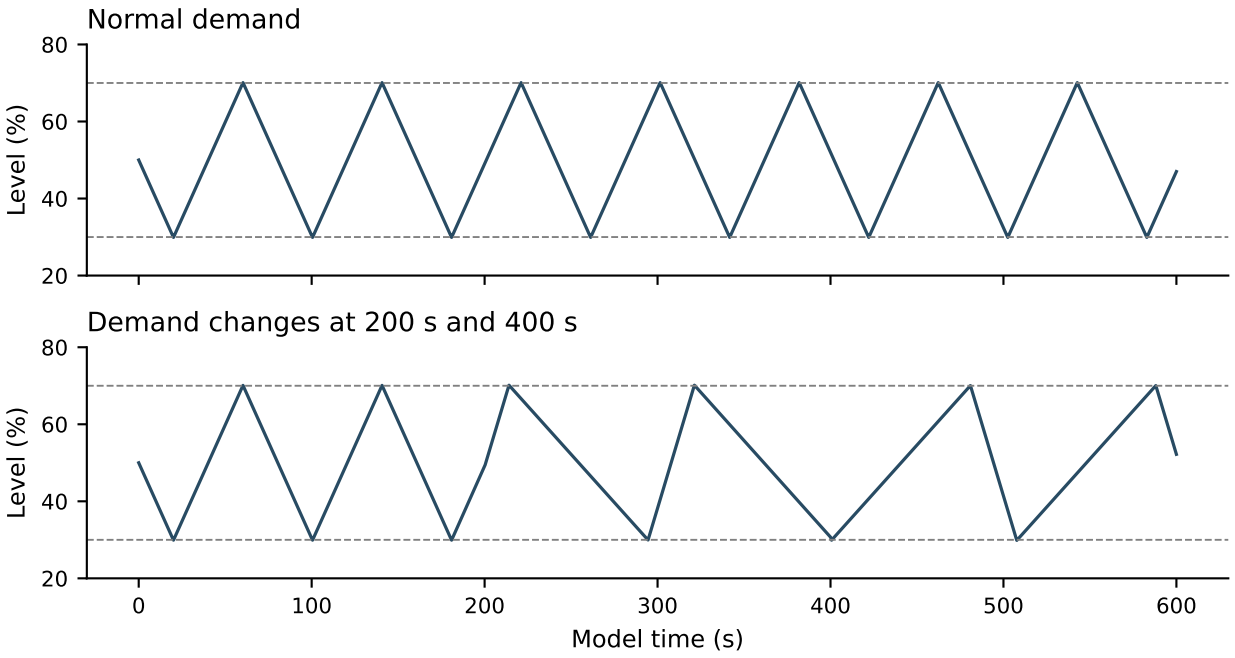


Fig. 1. Recorded native C-model level traces. Dashed lines show the 30% and 70% thresholds. The upper trace uses constant demand; the lower trace changes demand at 200 and 400 seconds.

TABLE I
RECORDED NATIVE C TANK RESULTS

Case	Period (ms)	Minimum (%)	Maximum (%)
Normal, 600 s	100	29.90	70.10
Demand changes	100	29.85	70.15
Demand above inlet	100	0.00	50.00
Valve forced open	100	40.00	100.00
Normal, 3600 s	100	29.90	70.10
Sampling sweep	50	29.95	70.05
Sampling sweep	100	29.90	70.10
Sampling sweep	500	29.50	70.50

IV. RESULTS AND ANALYSIS

Table I shows the measured limits. The slight overshoot is expected: the valve retains its state at the threshold and changes only after the next sampled value crosses it. Longer sampling intervals therefore allow more water to enter or leave before the next decision. Demand changes also alter that step size. The normal 100 ms run stayed within the planned 29.8–70.2% allowance, and the demand case did too.

The 600-second ARM feedback run matched the normal native result: 29.9–70.1%, with a final volume of 47 L. The 3600-second native run checked 36001 samples without underflow or overflow. Figure 1 shows how the demand changes alter the slopes while the threshold rule stays the same.

The shortage case reached zero even with the valve open because 3 L/s demand exceeds the 2 L/s inlet. The stuck-open case reached 100 L and continued to overflow while the controller commanded closure. These are modeled physical limits, not incorrect threshold decisions. Clamping the volume alone would have hidden them, which is why the raw-volume flags are recorded.

The host suites passed in GitHub Actions on Windows x86-64 and ARM64, macOS Intel and Apple Silicon, and Linux x86-64 and ARM64. Compiler target triples confirm native builds for those processors. AddressSanitizer and UndefinedBehaviorSanitizer passed on both architectures of macOS and Linux. Windows sanitizer execution is not claimed. Local Windows GCC lacked its UBSan runtime; CI used supported Unix runtimes for those checks.

The ARM build used GNU Arm GCC 15.2.1 and Renode 1.17.0. The size utility reported 7448 bytes of text, 12 bytes of initialized data, and 4660 bytes of BSS including the reserved sensor section. The linker reserves at least 4 KiB for the stack. This is a reservation, not a measured maximum stack depth.

Cross-platform testing exposed a reserved Windows fixture name and a C-library difference in zero-size line buffering. I renamed the fixture without changing its bytes and used explicit output flushing. The first emulator attempts also exposed console-log interleaving and a shared history-file collision between concurrent instances. The adapter now suppresses routine monitor logs, and the guide requires sequential Renode runs. These failed attempts were tool/adapter failures; I used the subsequent completed runs for the results above.

V. USER GUIDE

The accompanying `USER_GUIDE.txt` gives the complete commands. Install Python 3.9 or newer and a native GCC or Clang compiler, then run the fixture generator, baseline

runner, runtime tests, and tank tests from the extracted project directory. For example:

```
python Scripts/test_runtime.py -cc gcc
python Scripts/test_tank.py -cc gcc
```

On macOS or Linux, `python3` and `-cc cc` are usual choices. Windows Clang needs the matching Visual Studio developer environment. Each run creates a separate results folder and exits nonzero if a check fails.

For firmware, run `Scripts/build.py -firmware` with GNU Arm Embedded GCC, then `Scripts/test_firmware.py` with Renode and the Arm `nm` utility. The separate `test_renode_tank.py` adapter also needs the built native C model. Run one emulator adapter at a time. The demonstration command generator writes an inspectable LF-terminated command file.

VI. Goals Reached and Limits

The project implements repeatable hysteresis control, sensor validation, bounded command handling, authentication through Mbed TLS, and file-driven tank feedback. The executed tests support those goals on the stated native and emulated targets. The source, commands, fixtures, traces, and result summaries make the observations reproducible.

The remaining limits are explicit. Physical hardware, ADC behavior, electrical faults, secure key provisioning, encryption, replay protection, and actuator-failure detection are outside this implementation. The complete tank matrix ran natively; Renode ran one normal closed-loop case plus the sensor and serial-load tests. Emulated timing does not prove physical timing. Those boundaries keep the final claims tied to what I actually ran.

References

[1] F. Burns, "CYCS 3240 project source and evidence." [Online]. Available: https://github.com/grandAdmiralThrawn777/3240BigProject

[2] STMicroelectronics, "RM0090 STM32F4 reference manual." [Online]. Available: https://www.st.com/resource/en/reference_manual/rm0090-stm32f4xx-reference-manual-stmicroelectronics.pdf

[3] Renode, "Monitor and script syntax." [Online]. Available: https://renode.readthedocs.io/en/latest/basic/monitor-syntax.html

[4] Mbed TLS, "Release 3.6.7." [Online]. Available: https://github.com/Mbed-TLS/mbedtls/releases/tag/mbedtls-3.6.7

[5] SEI, "CERT C Coding Standard." [Online]. Available: https://cmu-sei.github.io/secure-coding-standards/sei-cert-c-coding-standard/

[6] M. Nystrom, "Identifiers and Test Vectors for HMAC-SHA-224, HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512," RFC 4231, Dec. 2005.

[7] IEEE, "IEEEtran class, version 1.8b." [Online]. Available: https://ctan.org/pkg/ieeetran